

CustomedViews.swift 实现详解

CustomedViews.swift 实现详解

一、整体介绍

CustomedViews.swift 是项目的 “UI 组件工具箱”，封装各类高频复用的界面组件，将常见 UI 功能标准化为 “可组装零件”，支持跨页面复用，核心价值是 “减少重复代码、统一界面风格、提升开发效率”。

二、主要组件拆解

文件包含 6 类核心组件，覆盖 “基础工具、数据展示、交互操作” 场景，各组件职责独立且可组合使用：

1. 基础工具类（底层通用功能）

为所有视图提供 “基础操作能力”，类似 “螺丝、扳手” 等通用工具，支撑上层组件实现：

- **圆角形状工具**：支持为视图 “指定特定角设置圆角”（如仅顶部两角圆角），而非默认的四角全圆角，适配卡片、弹窗等场景；
- **视图扩展工具**：通过 View 扩展，添加快捷方法（如 `cornerRadius(_:corners:)`），简化 “设置部分圆角” 的调用代码；
- **键盘隐藏工具**：提供 `hideKeyboard()` 方法，支持用户点击界面空白处时自动收起键盘，避免输入框被键盘遮挡。

2. 任务重排日期选择器（日期调整组件）

专门用于 “修改任务日期” 的弹窗组件，支持精确调整任务时间，核心功能如下：

- **界面元素**：包含日历视图（选择日期）、时间选择器（调整开始 / 结束时间）、确认 / 取消按钮；
- **交互逻辑**：
 - 用户选择新日期 / 时间后，点击 “确认” 按钮，通过回调函数将新时间传递给父组件；

- 点击“取消”按钮，关闭弹窗，不传递任何数据；
- **适用场景：**任务列表中“调整任务截止日期”“修改任务开始时间”等操作。

3. 标签圆形卡片（标签展示组件）

圆形风格的标签展示组件，用于标记任务分类（如“工作”“学习”标签），核心功能如下：

- **视觉样式：**圆形背景（自定义颜色）、白色文字，带轻微阴影，提升层次感；
- **交互效果：**点击时触发轻微缩放动画（如缩小至 0.95 倍再恢复），提供触觉反馈；
- **特殊能力：**支持长按拖动，可用于“标签排序”“任务标签调整”等场景；
- **数据绑定：**接收“标签名称”“标签颜色”参数，由父组件传入，无内部数据存储。

4. 通用任务列表视图（任务展示与操作组件）

全功能任务列表组件，支持“展示、操作、多选”等核心需求，是项目中任务管理的核心视图：

- **展示功能：**每行显示单个任务的“标题、截止时间、标签（标签圆形卡片）”，未完成 / 已完成任务用不同样式区分；
- **交互操作：**
 - 左右滑动：支持左滑删除任务、右滑标记任务完成 / 未完成；
 - 多选模式：长按任务进入多选模式，可批量删除、批量修改标签；
- **状态处理：**
 - 加载状态：数据加载中显示 `ProgressView`（转圈圈）；
 - 空状态：无任务时显示“暂无任务”提示文字与图标；
- **数据交互：**通过 `@Binding` 接收任务列表数据，用户操作（删除、移动）后同步更新绑定数据，无需手动回调。

5. 任务详情编辑对话框（任务编辑组件）

弹窗式任务编辑组件，用于查看 / 修改单个任务的完整信息，核心功能如下：

- **展示与编辑项：**包含“任务标题（输入框）、截止时间（日期选择器）、标签（标签选择器）、任务描述（文本框）”；
- **交互逻辑：**
 -

点击“保存”按钮：验证输入合法性（如标题非空），通过回调传递更新后的任务数据，关闭弹窗；

- 点击“取消”按钮：放弃编辑，关闭弹窗；

- **状态处理**：保存过程中显示加载指示器（菊花图），禁用按钮，避免重复提交；

- **依赖组件**：内部集成“任务重排日期选择器”（选择截止时间）、“标签圆形卡片”（展示已选标签）。

6. 日期导航栏（日期切换组件）

顶部导航栏组件，用于“日期 / 月份切换”，常见于任务列表顶部、日历视图顶部，核心功能如下：

- **显示内容**：居中显示当前日期（如“2024-05-20”）或当前月份（如“2024 年 5 月”）；

- **操作按钮**：

- 左右箭头：点击切换“上一天 / 下一天”或“上一月 / 下一月”，具体逻辑由父组件定义；

- “今天”按钮：点击快速跳转到当前日期 / 月份，无需逐步切换；

- **扩展功能**：点击居中日期，弹出完整日历视图，支持直接选择任意日期，选择后同步更新导航栏显示。

三、数据获取方式

组件不直接与“网络 / 数据库”交互，仅专注于“数据展示”与“用户操作反馈”，数据来源与传递通过以下 3 种方式：

1. **父组件传递**：核心数据（如任务列表、标签信息、当前日期）由使用组件的父视图通过参数传入（如 `@Binding let` 常量），组件仅负责渲染，不修改原始数据；

2. **回调函数反馈**：用户操作（如修改日期、删除任务、保存编辑）后，组件通过闭包（回调函数）将结果（如新日期、更新后的任务数据）传递给父组件，由父组件处理后续逻辑（如网络请求、本地存储）；

3. **内部临时状态**：组件内部通过 `@State` 管理临时数据（如输入框草稿、多选模式选中状态），这些数据仅在组件内部生效，不影响外部，组件销毁后状态重置。

四、界面刷新逻辑

依赖 SwiftUI 响应式特性，结合“状态管理 + 手动触发”，实现界面实时更新，核心逻辑如下：

- 1.

状态驱动自动刷新：组件通过 `@State`（内部临时状态）、`@Binding`（外部传递数据）标记关键数据，数据变化时，SwiftUI 自动重新渲染依赖该数据的 UI 元素（如任务列表数据更新后，列表自动刷新）；

2. **手动触发刷新：**通过调用“状态更新方法”（如 `updateSelectedTags(_:)`）或修改状态变量（如 `isMultiSelectMode = true`），手动触发界面特定部分刷新；

3. **通知机制适配：**监听系统通知（如 `UIResponder.keyboardWillShowNotification`），收到“键盘弹出”通知时，调整组件布局（如向上偏移），避免输入框被遮挡；

4. **键盘处理优化：**集成“键盘隐藏工具”，点击空白处触发 `hideKeyboard()`，同时监听键盘收起通知，自动恢复界面原始位置。

五、组件之间的关系

组件采用“分层组合”设计，底层组件支撑上层组件，上层组件可嵌套使用底层组件，形成灵活的组合关系：

- **基础工具类 → 所有组件：**“圆角形状工具”“键盘隐藏工具”被其他所有组件依赖，用于设置样式、处理键盘；
- **标签圆形卡片 → 通用任务列表视图 / 任务详情编辑对话框：**任务列表的每个任务项、编辑对话框的“已选标签区”，均嵌套使用“标签圆形卡片”展示标签；
- **任务重排日期选择器 → 任务详情编辑对话框：**编辑任务截止时间时，任务详情对话框弹出“任务重排日期选择器”，实现日期选择功能；
- **日期导航栏 → 通用任务列表视图：**任务列表顶部通常搭配“日期导航栏”，用于切换“按日 / 按月查看任务”。

六、简单总结

`CustomedViews.swift` 是项目的“UI 组件中枢”，核心特点可概括为：

维度	核心特点
功能定位	封装通用 UI 功能，提供“可复用、标准化”的组件，减少重复开发
数据交互	不直接处理数据存储 / 网络请求，仅通过“父组件传递 + 回调反馈”实现数据流转
刷新机制	

	以 SwiftUI 响应式状态为核心，结合手动触发与通知适配，确保界面实时更新
组合灵活性	组件分层设计，底层支撑上层，上层可嵌套使用，适配不同业务场景

通过这些组件，开发者可快速搭建 “任务列表、任务编辑、日期切换” 等核心界面，同时保证全 App UI 风格统一，降低维护成本。

（注：文档部分内容可能由 AI 生成）